

Intro

Zork is a text-based adventure game first released in 1977 by developers Tim Anderson, Marc Blank, Bruce Daniels, and Dave Lebling for the PDP-10 mainframe computer. . . . The player moves between the game's hundreds of locations and interacts with objects by typing commands in natural language that the game interprets. The program acts as a narrator, describing the player's location and the results of the player's commands. It has been described as the most famous piece of interactive fiction.¹

Theoretical Foundation

On a surface level, *Zork* does not appear much more complex than your previous implementations of text-based, input-output-driven programs. You enter some text, that text gets validated and interpreted, and finally an output gets printed based on your input.

```
West of House
You are standing in an open field west of a white house, with a boarded front door.
There is a small mailbox here.

> Open mailbox
Opening the small mailbox reveals a leaflet.

> Take leaflet
Taken.

> Read leaflet
"WELCOME TO ZORK!"
```

When thinking about on how to implement something based on the example above, problems start to arise very quickly:

- [Open mailbox] only makes sense if there is a mailbox.
- [Take leaflet] only makes sense if there is a leaflet.
- The leaflet can only be taken if the mailbox was opened previously.
- [Read leaflet] can only be done if it was taken previously.
- Things such as [Open leaflet] or [Read mailbox] don't make any sense.
- Being able to take things implies the existence of items and an inventory.
- Being at "West of House" implies that there are other locations. How do I get there?

It is advised to do some own research to deepen your understanding for how *Zork* looks like, works like and maybe even functions. For this project and prior to writing a single line of code, extra special attention should be given to planning:

- How does a player interact with the game?
- Can the game be broken down into smaller, more manageable components?
- Since this is a group effort (max. 2 people), who is responsible for which functionality?

It should also be mentioned that *Zork* only serves as an inspiration, mainly regarding how the game functions. As for the setting, it doesn't have to be an underground city rich in treasures and secrets. It might as well be your neighbourhood or an abandoned shopping mall. Feel free to come up with your own wacky scenarios! (Noser Young guidelines and code of conduct still apply)

¹ 'Zork' (2023) Wikipedia. Available at <https://en.wikipedia.org/wiki/Zork> (Accessed: 12 May 2023).

Implementation

Zork features close to 100 commands. And each of those commands can only be used at certain times or in certain places. For your *Zork*-like implementation, it is important to reduce the game of *Zork* to its essentials. Limit yourself to a select few commands like take, use, go, attack, and when creating your world to explore, be conservative with its size.

The following list features a few relevant components for your implementation. Whether these components are classes, methods, fields, or just useful information, is for you to find out.

Player	Item	Inventory
Where are they? How much health do they have?	What's its name? How many of a certain item do you have?	What's inside? Whom does the inventory belong to?
Room	Text Parser / Input Handler	Game / Game Controller
How does the room look? Can an item be used here? Which rooms are connected with each other and how? Does the room contain an item and / or an enemy?	What inputs are allowed? Which commands exist? Do certain commands need a parameter / second word? How are certain actions bound to certain commands?	How is the game played? Can the game be saved? How can the game be won or lost? Do things happen even without player influence?

Implementing your project with scalability in mind should be of high priority. This means that it shouldn't take a rewrite of half your code when you want to add a new item to the game later. Or needing to copy and paste multiple lines of code for adding a new room to explore.

As per the usual, your own features that deviate from the expected base features are allowed, but shouldn't take higher priority than the base features or even impact them in a negative way.

Tips

While previous challenges often made use of 2D arrays, it should be mentioned that for this one, 2D arrays explicitly mustn't be used. If a room were to be implemented as a class, then one room can have a reference to another room, or even multiple other rooms. These references (fields) to other room objects can then be accessed using appropriate methods. Furthermore, defining where an item is and where, what item can be used might also be the job of the room class.

Advanced Tasks

Before continuing here, make sure that your base implementation works and is also of good quality. Consider to black-box-test your program and request a code review. Furthermore, these advanced tasks are separate from each other. So, feel free to tackle them in any order you like.

- **Backtracking:** Command that sends you to the previously visited room (Hint: Stack).
- **Saving:** Functionality to persist your game progress (Hint: File I/O).
- **Play Time:** Functionality that keeps track of your play time.
- **Map:** Item that generates a visual representation of your map.